Process & Decision Documentation

Side Quests

A 2D pixel platformer where you play as a fox racing against the clock. The objective is to navigate a multi-tiered tile-based level to find a randomly spawning treasure chest. The game features a two-level progression system: clear Level 1 within 30 seconds to unlock a more challenging Level 2, where the timer is cut down to just 15 seconds. It also includes a debug screen for testing physics and game states.

If GenAI was used (keep each response as brief as possible):

Date Used: Mar 22, 2026

Tool Disclosure: Gemini 3.1 Pro

Purpose of Use: Implements the developer debug screen in the Week 6 game and add another game level.

Summary of Interaction: I provided my base code and a visual reference from a lecture slide. Gemini generated the logic for the debug toggles, HUD overlay, and state-machine transitions. I then prompted it to refine the UI layout and add Level 2.

Human Decision Point(s): I decided which specific debug features made sense for my game (stats, collisions, invincibility) rather than copying the reference image exactly. I also designed the game mechanics for Level 2.

Integrity & Verification Note: All GenAI-produced code was reviewed and playtested within a local live server to ensure it correctly hooked into p5play's engine and my existing state machine without breaking movement or collisions.

Scope of GenAI Use: Code generation for new feature logic (UI drawing, boolean toggles, state machine expansion).

Limitations or Misfires: The initial debug UI box generated was too small to contain the text properly, requiring a specific follow-up prompt to manually adjust the rect() dimensions and padding.

Summary of Process (Human + Tool)

I started with the Side Quest 6. To make the debug screen, I used Gemini to build an interactive debug screen. By providing a lecture screenshot as inspiration, I guided the AI to create a custom menu toggled by the backtick key. Once the debugging foundation was solid, I shifted focus to game flow. I conceptualized a two-tier level system and used GenAI to help expand my state machine (WIN_LEVEL vs WIN_GAME), ensuring the game resets properly while dynamically adjusting the time limit based on the current level.

Decision Points & Trade-offs

I designed the gameplay mechanics for Level 2 and determined how to increase the game's difficulty. I used iteration and playtesting to evaluate whether the mechanics were effective.

Verification & Judgement

I turned on the collision debug view (Option 2) to physically verify that my .resize() on the treasure chest asset matched the physical hitbox the player interacts with. I also heavily relied on the Invincibility toggle (Option 3) to rapidly test jump arcs without the pressure of the 30-second timer. The level transition was verified by playing through naturally to ensure the timer correctly swapped to 15s upon starting Level 2.

Limitations, Dead Ends, or Open Questions

Limitation: The current debug menu overlays a large portion of the center screen. While fine for a pause menu, it can obstruct gameplay if left open while moving.

Appendix

Appendix: GenAI Transcript

Tool Used: Gemini 3.1 Pro **Date:** Mar 22, 2026

Prompt： Can you open this code and take a look at all the files? I need a debug screen for this game. *[uploaded index.html, jsconfig.json,sketch.js, style.css]*

Gemini: This looks like a fun side quest project! You have a solid foundation with a state machine, a camera system, and some clean pixel-perfect rendering logic.

To add a debug screen, we can introduce a toggle (like pressing the backtick ` key) that will turn on p5play's built-in sprite debugging (which shows hitboxes and velocities) and overlay a custom Heads-Up Display (HUD) showing useful under-the-hood stats like FPS, player coordinates, and grounded state.
Here are the changes you need to make to your **sketch.js** file.

Step 1: Add a Global Variable

At the top of your sketch.js file, add a boolean to keep track of whether the debug mode is active.
JavaScript

```
// Add this near your other global variables at the top
let showDebug = false;
```

Step 2: Add the Toggle Logic

Inside your draw() function, add a key press listener to flip the showDebug boolean. This can go right at the start of the draw() loop, just under background(69, 61, 79);.
JavaScript

```
function draw() {
  background(69, 61, 79);

  // --- DEBUG TOGGLE ---
  // Press the backtick key ( ` ) or tilde ( ~ ) to toggle debug mode
  if (kb.presses('`')) {
    showDebug = !showDebug;
    allSprites.debug = showDebug; // Toggles p5play's visual hitboxes
  }

  // ... (rest of your existing draw code)
```

Step 3: Draw the Debug HUD

At the very end of your draw() function, after you restore the player and sensor coordinates, add the code to render the debug text. We need to call camera.off() here so the text stays glued to the screen rather than moving with the game world.
JavaScript

```
  // ... (end of your current draw function)

  player.x = px;
  player.y = py;
  sensor.x = sx;
  sensor.y = sy;

  // --- DEBUG OVERLAY ---
  if (showDebug) {
    camera.off(); // Detach from world camera to draw UI on screen

    // Draw a semi-transparent background box for readability
    push();
    fill(0, 150);
```

```
    noStroke();
    rect(0, 0, 140, 90);

    // Draw the debug statistics
    fill(255);
    textFont('monospace'); // Uses standard monospace for clean alignment
    textSize(10);
    textAlign(LEFT, TOP);

    // Check grounded state for the debug text
    let grounded = sensor.overlapping(ground) ||
                   sensor.overlapping(platformsL) ||
                   sensor.overlapping(platformsR);

    let debugText =
      `FPS: ${Math.round(frameRate())}\n` +
      `State: ${gameState}\n` +
      `Time: ${timeLeft}s\n` +
      `Pos: x:${Math.round(player.x)} y:${Math.round(player.y)}\n` +
      `Vel: x:${player.vel.x.toFixed(1)} y:${player.vel.y.toFixed(1)}\n` +
      `Grounded: ${grounded}`;

    text(debugText, 5, 5);
    pop();

    camera.on(); // Turn camera back on for the next frame
  }
} // End of draw() function
```

What This Does:

1. **allSprites.debug = showDebug;**: This is a powerful p5play feature. When true, it automatically draws the green/red collision borders around your fox, the ground tiles, the treasure, and the invisible sensor, making it incredibly easy to fix hitbox issues.
2. **Text Overlay**: Shows exactly what the math is doing in real-time. Tracking player.vel.y and your grounded boolean is particularly helpful for debugging jumping/falling mechanics.

Prompt：Can you give me all the code that needs to be changed?

Gemini: Here is the complete, updated **sketch.js** file. You can simply copy this entire block of code and replace the contents of your current sketch.js file.
I've added comments tagged with // --- NEW: DEBUG ... --- so you can easily spot exactly where the additions were made.

JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
```

```
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack
    ` (Backtick)                  Toggle Debug Mode // <-- Added to docs
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg, treasureImg;

// UI variables for our HTML overlay
let uiDiv, endDiv;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img,
  groundTile2Img,
  platforTileLImg,
  platforTileRImg,
  wallTileLImg,
  wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// --- NEW: DEBUG VARIABLE ---
let showDebug = false;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
  { c: 5, r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4, r: 4 },
```

```
  { c: 15, r: 4 },
  { c: 21, r: 4 },
];

// --- TILE MAP ---
let level = [
  "                g                  ", // row  0
  "                                   ", // row  1
  "            LggR                   ", // row  2
  "    LR   LgR         LR            ", // row  3: upper platforms
  "                                   ", // row  4
  "   LgggR      LR   LgR             ", // row  5: mid platforms
  "        LgR           g   LggggR   ", // row  6: walls + low platform
  "            LgR                    ", // row  7
  "                            LggR", // row  8
  "        LgR              LR  LR  [dd]", // row  9: mid-right platform
  "        [d]       gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  treasureImg = loadImage("assets/treasure.png");
```

```
  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- HTML UI OVERLAY ---
  uiDiv = createDiv(
    '<div style="font-family: \'Times New Roman\', serif; font-size: 32px; font-weight: bold;">Find the
treasure before time runs out<br><span id="time-span" style="font-size: 42px;">30s</span></div>',
  );
  uiDiv.position(0, 0);
  uiDiv.style("width", "100%");
  uiDiv.style("background", "rgba(0, 0, 0, 0.6)");
  uiDiv.style("color", "white");
  uiDiv.style("text-align", "center");
  uiDiv.style("padding", "15px 0");
  uiDiv.style("pointer-events", "none");

  endDiv = createDiv(
    '<div id="end-title" style="font-family: \'Times New Roman\', serif; font-size: 64px; font-weight:
bold;"></div><div style="font-family: \'Times New Roman\', serif; font-size: 32px; margin-top: 15px;
color: white;">Press R to restart</div>',
  );
  endDiv.position(0, 0);
  endDiv.style("width", "100%");
  endDiv.style("height", "100%");
  endDiv.style("background", "rgba(0, 0, 0, 0.8)");
  endDiv.style("display", "none");
  endDiv.style("flex-direction", "column");
  endDiv.style("justify-content", "center");
  endDiv.style("align-items", "center");
  endDiv.style("pointer-events", "none");

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundTile2Img;
  groundDeep.tile = "d";
```

```
platformsL = new Group();
platformsL.physics = "static";
platformsL.img = platformTileLImg;
platformsL.tile = "L";

platformsR = new Group();
platformsR.physics = "static";
platformsR.img = platformTileRImg;
platformsR.tile = "R";

wallsL = new Group();
wallsL.physics = "static";
wallsL.img = wallTileLImg;
wallsL.tile = "[";

wallsR = new Group();
wallsR.physics = "static";
wallsR.img = wallTileRImg;
wallsR.tile = "]";

new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -8;
player.addAnis(playerAnis);

player.ani = "idle";
player.w = 18;
player.h = 12;
player.friction = 0;
player.bounciness = 0;

// --- TREASURE ---
treasureImg.resize(24, 0);

treasure = new Sprite();
treasure.img = treasureImg;
treasure.w = 16;
treasure.h = 16;
treasure.collider = "static";
treasure.overlaps(player);

resetLevel(false);

// --- GROUND SENSOR ---
sensor = new Sprite();
sensor.x = player.x;
```

```
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  // --- NEW: DEBUG TOGGLE LOGIC ---
  if (kb.presses('`')) {
    showDebug = !showDebug;
    allSprites.debug = showDebug;
  }

  if (gameState === "PLAYING") {
    world.step();

    // --- TIMER LOGIC ---
    if (!timerStarted) {
      if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
        timerStarted = true;
        lastTick = millis();
      }
    } else {
      if (millis() - lastTick >= 1000 && timeLeft > 0) {
        timeLeft--;
        lastTick = millis();
        document.getElementById("time-span").innerText = timeLeft + "s";
      }
    }

    // --- CAMERA ---
    camera.width = VIEWW;
    camera.height = VIEWH;

    let targetX = constrain(
      player.x,
      VIEWW / 2,
      LEVELW - VIEWW / 2 - TILE_W / 2,
```

```
);
let targetY = constrain(
  player.y,
  VIEWH / 2 - TILE_H * 2,
  LEVELH - VIEWH / 2 - TILE_H,
);

camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

// --- PLAYER CONTROLS ---
let grounded =
  sensor.overlapping(ground) ||
  sensor.overlapping(platformsL) ||
  sensor.overlapping(platformsR);

// -- ATTACK INPUT --
if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4.5;
  jumpSound.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
```

```
      isMoving = true;
    } else if (kb.pressing("right")) {
      player.vel.x = 1.5;
      player.mirror.x = false;
      isMoving = true;
    }
  }

  // --- RUNNING AUDIO LOGIC ---
  if (grounded && isMoving && !attacking) {
    if (!runSound.isPlaying()) {
      runSound.loop();
    }
  } else {
    if (runSound.isPlaying()) {
      runSound.stop();
    }
  }

  // --- PLAYER BOUNDS ---
  player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

  // --- GAME END/WIN STATES ---
  if (player.overlaps(treasure)) {
    gameState = "WIN";
    if (runSound.isPlaying()) runSound.stop();

    // Update HTML UI
    uiDiv.hide();
    endDiv.style("display", "flex");
    document.getElementById("end-title").innerText = "YOU WIN";
    document.getElementById("end-title").style.color = "#00ff00";
  } else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {
    if (player.y > LEVELH + TILE_H * 3) {
      fallSound.play();
    }
    gameState = "LOSE";
    if (runSound.isPlaying()) runSound.stop();

    // Update HTML UI
    uiDiv.hide();
    endDiv.style("display", "flex");
    document.getElementById("end-title").innerText = "YOU LOSE";
    document.getElementById("end-title").style.color = "#ff0000";
  }
} else {
  // Listen for restart when the game is over
  if (kb.presses("r")) {
    resetLevel(false);
  }
}

// --- BACKGROUNDS ---
```

```
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- PIXEL SNAP ---
const px = player.x,
  py = player.y;
const sx = sensor.x,
  sy = sensor.y;

player.x = Math.round(player.x);
player.y = Math.round(player.y);
sensor.x = Math.round(sensor.x);
sensor.y = Math.round(sensor.y);

allSprites.draw();

player.x = px;
player.y = py;
sensor.x = sx;
sensor.y = sy;

// --- NEW: DEBUG HUD DRAWING ---
if (showDebug) {
  camera.off(); // Detach from world camera to draw UI on screen

  // Draw a semi-transparent background box for readability
  push();
  fill(0, 150);
  noStroke();
  rect(0, 0, 140, 95); // Background box

  // Draw the debug statistics
  fill(255);
  textFont('monospace'); // Uses standard monospace for clean alignment
  textSize(10);
  textAlign(LEFT, TOP);

  // Re-check grounded state specifically for the debug text
  let grounded = sensor.overlapping(ground) ||
                 sensor.overlapping(platformsL) ||
```

```
                sensor.overlapping(platformsR);

    let debugText =
      `FPS: ${Math.round(frameRate())}\n` +
      `State: ${gameState}\n` +
      `Time: ${timeLeft}s\n` +
      `Pos: x:${Math.round(player.x)} y:${Math.round(player.y)}\n` +
      `Vel: x:${player.vel.x.toFixed(1)} y:${player.vel.y.toFixed(1)}\n` +
      `Grounded: ${grounded}`;

    text(debugText, 5, 5);
    pop();

    camera.on(); // Turn camera back on for the next frame
  }
}

function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
    fallSound.play();
  }

  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  // Reset HTML UI
  if (uiDiv) {
    uiDiv.show();
    document.getElementById("time-span").innerText = timeLeft + "s";
  }
  if (endDiv) endDiv.style("display", "none");

  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(
    1,
    Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight / VIEWH)),
  );
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Prompt：Can you add a debug screen like this? *[uploaded week 9 slides]*

Gemini: That is a great reference image! A dedicated, interactive debug menu is much better for testing than a simple text overlay.

Since your game doesn't have "boars" or complex win conditions, I've adapted the menu items to fit your current mechanics perfectly. I've also hooked it up to the VT323 pixel font you already have linked in your HTML file to give it that retro aesthetic from your lecture slide.
Here is the fully updated **sketch.js** file.

What's included in this menu:

- **Press ` (Backtick) to open/close the menu.**
- **[1] Show HUD Stats:** Toggles the technical text overlay we added previously.
- **[2] Show Collisions:** Toggles p5play's built-in green/red hitboxes.
- **[3] Player Invincible:** Freezes the timer so you don't run out of time, and if you fall down a pit, you are safely teleported back to the start instead of getting the "Game Over" screen.

JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack
    ` (Backtick)                  Toggle Debug Menu
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg, treasureImg;

// UI variables for our HTML overlay
let uiDiv, endDiv;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
```

```
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img,
  groundTile2Img,
  platforTileLImg,
  platforTileRImg,
  wallTileLImg,
  wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// --- DEBUG MENU VARIABLES ---
let debugMenuOpen = false;
let dbgShowStats = false;
let dbgShowCollisions = false;
let dbgInvincible = false;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
  { c: 5, r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4, r: 4 },
  { c: 15, r: 4 },
  { c: 21, r: 4 },
];

// --- TILE MAP ---
let level = [
  "                  g                  ", // row  0
  "                                     ", // row  1
  "              LggR                   ", // row  2
  "     LR   LgR         LR             ", // row  3: upper platforms
  "                                     ", // row  4
  "   LgggR      LR   LgR               ", // row  5: mid platforms
  "        LgR           g   LggggR     ", // row  6: walls + low platform
  "             LgR                     ", // row  7
  "                              LggR", // row  8
  "         LgR              LR  LR  [dd]", // row  9: mid-right platform
  "         [d]        gggg          [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];
```

```
// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  treasureImg = loadImage("assets/treasure.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- HTML UI OVERLAY ---
  uiDiv = createDiv(
    '<div style="font-family: \'Times New Roman\', serif; font-size: 32px; font-weight: bold;">Find the
treasure before time runs out<br><span id="time-span" style="font-size: 42px;">30s</span></div>',
  );
  uiDiv.position(0, 0);
```

```
  uiDiv.style("width", "100%");
  uiDiv.style("background", "rgba(0, 0, 0, 0.6)");
  uiDiv.style("color", "white");
  uiDiv.style("text-align", "center");
  uiDiv.style("padding", "15px 0");
  uiDiv.style("pointer-events", "none");

  endDiv = createDiv(
    '<div id="end-title" style="font-family: \'Times New Roman\', serif; font-size: 64px; font-weight:
bold;"></div><div style="font-family: \'Times New Roman\', serif; font-size: 32px; margin-top: 15px;
color: white;">Press R to restart</div>',
  );
  endDiv.position(0, 0);
  endDiv.style("width", "100%");
  endDiv.style("height", "100%");
  endDiv.style("background", "rgba(0, 0, 0, 0.8)");
  endDiv.style("display", "none");
  endDiv.style("flex-direction", "column");
  endDiv.style("justify-content", "center");
  endDiv.style("align-items", "center");
  endDiv.style("pointer-events", "none");

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundTile2Img;
  groundDeep.tile = "d";

  platformsL = new Group();
  platformsL.physics = "static";
  platformsL.img = platformTileLImg;
  platformsL.tile = "L";

  platformsR = new Group();
  platformsR.physics = "static";
  platformsR.img = platformTileRImg;
  platformsR.tile = "R";

  wallsL = new Group();
  wallsL.physics = "static";
  wallsL.img = wallTileLImg;
  wallsL.tile = "[";

  wallsR = new Group();
  wallsR.physics = "static";
  wallsR.img = wallTileRImg;
  wallsR.tile = "]";
```

```
  new Tiles(level, 0, 0, TILE_W, TILE_H);

  // --- PLAYER ---
  player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
  player.spriteSheet = playerImg;
  player.rotationLock = true;
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
  player.anis.offset.y = -8;
  player.addAnis(playerAnis);

  player.ani = "idle";
  player.w = 18;
  player.h = 12;
  player.friction = 0;
  player.bounciness = 0;

  // --- TREASURE ---
  treasureImg.resize(24, 0);

  treasure = new Sprite();
  treasure.img = treasureImg;
  treasure.w = 16;
  treasure.h = 16;
  treasure.collider = "static";
  treasure.overlaps(player);

  resetLevel(false);

  // --- GROUND SENSOR ---
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;

  // --- BACKGROUND  ---
  bgLayers = [
    { img: bgFarImg, speed: 0.2 },
    { img: bgMidImg, speed: 0.4 },
    { img: bgForeImg, speed: 0.6 },
  ];

  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);
```

```
// --- DEBUG MENU TOGGLE LOGIC ---
if (kb.presses('`')) {
  debugMenuOpen = !debugMenuOpen;
}

// Handle Input for Debug Menu
if (debugMenuOpen) {
  if (kb.presses('1')) {
    dbgShowStats = !dbgShowStats;
  }
  if (kb.presses('2')) {
    dbgShowCollisions = !dbgShowCollisions;
    allSprites.debug = dbgShowCollisions;
  }
  if (kb.presses('3')) {
    dbgInvincible = !dbgInvincible;
  }
}

if (gameState === "PLAYING") {
  world.step();

  // --- TIMER LOGIC (Modified for Debug Invincibility) ---
  if (!timerStarted) {
    if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
      timerStarted = true;
      lastTick = millis();
    }
  } else {
    if (millis() - lastTick >= 1000) {
      if (!dbgInvincible && timeLeft > 0) {
        timeLeft--;
      }
      lastTick = millis();
      // Show infinity symbol if invincible
      let timeDisplay = dbgInvincible ? "∞" : timeLeft + "s";
      document.getElementById("time-span").innerText = timeDisplay;
    }
  }

  // --- CAMERA ---
  camera.width = VIEWW;
  camera.height = VIEWH;

  let targetX = constrain(
    player.x,
    VIEWW / 2,
    LEVELW - VIEWW / 2 - TILE_W / 2,
  );
  let targetY = constrain(
    player.y,
    VIEWH / 2 - TILE_H * 2,
```

```
  LEVELH - VIEWH / 2 - TILE_H,
);

camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

// --- PLAYER CONTROLS ---
let grounded =
  sensor.overlapping(ground) ||
  sensor.overlapping(platformsL) ||
  sensor.overlapping(platformsR);

// -- ATTACK INPUT --
if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4.5;
  jumpSound.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
```

```
      isMoving = true;
    }
  }

  // --- RUNNING AUDIO LOGIC ---
  if (grounded && isMoving && !attacking) {
    if (!runSound.isPlaying()) {
      runSound.loop();
    }
  } else {
    if (runSound.isPlaying()) {
      runSound.stop();
    }
  }

  // --- PLAYER BOUNDS ---
  player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

  // --- GAME END/WIN STATES ---
  if (player.overlaps(treasure)) {
    gameState = "WIN";
    if (runSound.isPlaying()) runSound.stop();

    uiDiv.hide();
    endDiv.style("display", "flex");
    document.getElementById("end-title").innerText = "YOU WIN";
    document.getElementById("end-title").style.color = "#00ff00";

  } else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {

    // If invincible, prevent death and teleport back to start if they fall
    if (dbgInvincible && player.y > LEVELH + TILE_H * 3) {
      player.x = FRAME_W;
      player.y = PLAYER_START_Y;
      player.vel.x = 0;
      player.vel.y = 0;
      fallSound.play(); // Play sound but don't lose
    } else if (!dbgInvincible) {
      if (player.y > LEVELH + TILE_H * 3) {
        fallSound.play();
      }
      gameState = "LOSE";
      if (runSound.isPlaying()) runSound.stop();

      uiDiv.hide();
      endDiv.style("display", "flex");
      document.getElementById("end-title").innerText = "YOU LOSE";
      document.getElementById("end-title").style.color = "#ff0000";
    }
  }
} else {
  // Listen for restart when the game is over
  if (kb.presses("r")) {
```

```
    resetLevel(false);
  }
}

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- PIXEL SNAP ---
const px = player.x,
  py = player.y;
const sx = sensor.x,
  sy = sensor.y;

player.x = Math.round(player.x);
player.y = Math.round(player.y);
sensor.x = Math.round(sensor.x);
sensor.y = Math.round(sensor.y);

allSprites.draw();

player.x = px;
player.y = py;
sensor.x = sx;
sensor.y = sy;

// --- HUD STATS OVERLAY ---
if (dbgShowStats) {
  camera.off();
  push();
  fill(0, 150);
  noStroke();
  rect(0, 0, 140, 95);

  fill(255);
  textFont('monospace');
  textSize(10);
  textAlign(LEFT, TOP);

  let grounded = sensor.overlapping(ground) ||
```

```
            sensor.overlapping(platformsL) ||
            sensor.overlapping(platformsR);

  let debugText =
    `FPS: ${Math.round(frameRate())}\n` +
    `State: ${gameState}\n` +
    `Time: ${dbgInvincible ? '∞' : timeLeft + 's'}\n` +
    `Pos: x:${Math.round(player.x)} y:${Math.round(player.y)}\n` +
    `Vel: x:${player.vel.x.toFixed(1)} y:${player.vel.y.toFixed(1)}\n` +
    `Grounded: ${grounded}`;

  text(debugText, 5, 5);
  pop();
  camera.on();
}

// --- FULL DEBUG MENU OVERLAY ---
if (debugMenuOpen) {
  camera.off();

  push();
  // Dark translucent background for the menu
  rectMode(CENTER);
  fill(0, 0, 0, 220);
  stroke(100);
  strokeWeight(2);
  rect(VIEWW / 2, VIEWH / 2, 200, 140, 8);

  // Menu Title
  textAlign(CENTER, CENTER);
  fill(255, 255, 0); // Yellow title
  noStroke();
  textFont('"VT323", monospace');
  textSize(24);
  text("DEBUG MENU", VIEWW / 2, VIEWH / 2 - 45);

  // Menu Options
  textSize(14);
  textAlign(LEFT, CENTER);
  let startX = VIEWW / 2 - 80;
  let startY = VIEWH / 2 - 15;
  let spacing = 20;

  fill(255);
  text(`[1] Show HUD Stats:    ${dbgShowStats ? "ON" : "OFF"}`, startX, startY);
  text(`[2] Show Collisions:   ${dbgShowCollisions ? "ON" : "OFF"}`, startX, startY + spacing);
  text(`[3] Player Invincible: ${dbgInvincible ? "ON" : "OFF"}`, startX, startY + spacing * 2);

  // Footer Instructions
  textAlign(CENTER, CENTER);
  fill(150);
  textSize(12);
  text("Press 1-3 to toggle, ` to close", VIEWW / 2, VIEWH / 2 + 50);
```

```
    pop();
    camera.on();
  }
}

function resetLevel(playFallSound) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
    fallSound.play();
  }

  gameState = "PLAYING";
  timeLeft = 30;
  timerStarted = false;

  // Reset HTML UI
  if (uiDiv) {
    uiDiv.show();
    document.getElementById("time-span").innerText = dbgInvincible ? "∞" : timeLeft + "s";
  }
  if (endDiv) endDiv.style("display", "none");

  let spawn = random(spawnPoints);
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(
    1,
    Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight / VIEWH)),
  );
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```

Prompt：

1. Make the background of the debug screen a little longer so that it covers all the text

2. Add a “Level 1” at the top of the game, and give me a “Level 2.” For Level 2, mainly change the time from 30 seconds to 15 seconds; everything else remains the same.

[uploaded current game interface]

Gemini: That debug menu looks great in action!

To fix the background box size and implement your Level 1 / Level 2 progression, we just need to adjust the debug rect() dimensions and add a currentLevel variable that scales the time limit and handles transitioning between the win screens.
Here is the fully updated **sketch.js** file.

What's Changed:

1. **Debug Menu Background:** Increased the height and width of the debug menu's background rectangle and shifted it down slightly so the footer text fits perfectly inside.
2. **Level Progression:** * Added a currentLevel variable.
    - Updated the top HTML UI to display "Level 1" (or 2).
    - When you grab the treasure on Level 1, you get a "LEVEL 1 CLEAR! Press C to start Level 2" screen.
    - When you press 'C', the game resets but gives you 15 seconds instead of 30.
    - Winning Level 2 gives you the final "YOU WIN" screen. Pressing 'R' restarts back at Level 1.

JavaScript

```
/*
  Week 6 — Example 3: Expanded Tile-Based Level with Camera Follow, Fall Reset, and Scrolling Background

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack
    ` (Backtick)                  Toggle Debug Menu
*/

let player, sensor, treasure;
let bgLayers = [];
let playerImg, bgForeImg, bgMidImg, bgFarImg, treasureImg;

// UI variables for our HTML overlay
let uiDiv, endDiv;

// Audio variables
let runSound, jumpSound, fallSound;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
```

```
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep, platformsL, platformsR, wallsL, wallsR;
let groundTile1Img,
  groundTile2Img,
  platforTileLImg,
  platforTileRImg,
  wallTileLImg,
  wallTileRImg;

let attacking = false;
let attackFrameCounter = 0;

// --- GAME LOGIC VARIABLES ---
let gameState = "PLAYING";
let currentLevel = 1; // Track the current level
let maxTime = 30;     // Stores the starting time for the level
let timeLeft = 30;
let timerStarted = false;
let lastTick = 0;

// --- DEBUG MENU VARIABLES ---
let debugMenuOpen = false;
let dbgShowStats = false;
let dbgShowCollisions = false;
let dbgInvincible = false;

// Coordinate grids (column and row) targeting the upper platforms
let spawnPoints = [
  { c: 17, r: 1 },
  { c: 5, r: 2 },
  { c: 11, r: 2 },
  { c: 23, r: 2 },
  { c: 4, r: 4 },
  { c: 15, r: 4 },
  { c: 21, r: 4 },
];

// --- TILE MAP ---
let level = [
  "                  g                  ", // row  0
  "                                     ", // row  1
  "              LggR                   ", // row  2
  "     LR   LgR         LR             ", // row  3: upper platforms
  "                                     ", // row  4
  "   LgggR       LR   LgR              ", // row  5: mid platforms
  "        LgR           g   LggggR     ", // row  6: walls + low platform
  "             LgR                     ", // row  7
  "                              LggR", // row  8
  "         LgR              LR  LR  [dd]", // row  9: mid-right platform
```

```
  "          [d]        gggg           [dd]", // row 10: wall below mid-right platform
  "ggggg  gggggggg   gggggg  g gggggggggg", // row 11: surface ground WITH GAPS
  "ddddd  dddddddd   ddddddd    ddddddddddd", // row 12: deep ground
];

// --- LEVEL CONSTANTS ---
const TILE_W = 24;
const TILE_H = 24;

const FRAME_W = 32;
const FRAME_H = 32;

const LEVELW = TILE_W * level[0].length;
const LEVELH = TILE_H * level.length;

const VIEWTILE_W = 10;
const VIEWTILE_H = 8;
const VIEWW = TILE_W * VIEWTILE_W;
const VIEWH = TILE_H * VIEWTILE_H;

const PLAYER_START_Y = LEVELH - TILE_H * 4;
const GRAVITY = 10;

function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgFarImg = loadImage("assets/background_layer_1.png");
  bgMidImg = loadImage("assets/background_layer_2.png");
  bgForeImg = loadImage("assets/background_layer_3.png");
  groundTile1Img = loadImage("assets/groundTile.png");
  groundTile2Img = loadImage("assets/groundTileDeep.png");
  platformTileLImg = loadImage("assets/platformLC.png");
  platformTileRImg = loadImage("assets/platformRC.png");
  wallTileLImg = loadImage("assets/wallL.png");
  wallTileRImg = loadImage("assets/wallR.png");

  treasureImg = loadImage("assets/treasure.png");

  runSound = loadSound("assets/running.mp3");
  jumpSound = loadSound("assets/jumping.mp3");
  fallSound = loadSound("assets/falling.mp3");
}

function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  noSmooth();

  applyIntegerScale();
  window.addEventListener("resize", applyIntegerScale);

  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  // --- HTML UI OVERLAY ---
```

```
  uiDiv = createDiv(
    '<div style="font-family: \'Times New Roman\', serif; font-size: 32px; font-weight: bold;"><span
id="level-span" style="color: #FFD700;">Level 1</span><br>Find the treasure before time runs
out<br><span id="time-span" style="font-size: 42px;">30s</span></div>',
  );
  uiDiv.position(0, 0);
  uiDiv.style("width", "100%");
  uiDiv.style("background", "rgba(0, 0, 0, 0.6)");
  uiDiv.style("color", "white");
  uiDiv.style("text-align", "center");
  uiDiv.style("padding", "10px 0");
  uiDiv.style("pointer-events", "none");

  endDiv = createDiv(
    '<div id="end-title" style="font-family: \'Times New Roman\', serif; font-size: 64px; font-weight:
bold;"></div><div id="end-subtitle" style="font-family: \'Times New Roman\', serif; font-size: 32px;
margin-top: 15px; color: white;">Press R to restart</div>',
  );
  endDiv.position(0, 0);
  endDiv.style("width", "100%");
  endDiv.style("height", "100%");
  endDiv.style("background", "rgba(0, 0, 0, 0.8)");
  endDiv.style("display", "none");
  endDiv.style("flex-direction", "column");
  endDiv.style("justify-content", "center");
  endDiv.style("align-items", "center");
  endDiv.style("pointer-events", "none");

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundTile1Img;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundTile2Img;
  groundDeep.tile = "d";

  platformsL = new Group();
  platformsL.physics = "static";
  platformsL.img = platformTileLImg;
  platformsL.tile = "L";

  platformsR = new Group();
  platformsR.physics = "static";
  platformsR.img = platformTileRImg;
  platformsR.tile = "R";

  wallsL = new Group();
  wallsL.physics = "static";
  wallsL.img = wallTileLImg;
  wallsL.tile = "[";
```

```
wallsR = new Group();
wallsR.physics = "static";
wallsR.img = wallTileRImg;
wallsR.tile = "]";

new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER ---
player = new Sprite(FRAME_W, PLAYER_START_Y, FRAME_W, FRAME_H);
player.spriteSheet = playerImg;
player.rotationLock = true;
player.anis.w = FRAME_W;
player.anis.h = FRAME_H;
player.anis.offset.y = -8;
player.addAnis(playerAnis);

player.ani = "idle";
player.w = 18;
player.h = 12;
player.friction = 0;
player.bounciness = 0;

// --- TREASURE ---
treasureImg.resize(24, 0);

treasure = new Sprite();
treasure.img = treasureImg;
treasure.w = 16;
treasure.h = 16;
treasure.collider = "static";
treasure.overlaps(player);

resetLevel(false, 1);

// --- GROUND SENSOR ---
sensor = new Sprite();
sensor.x = player.x;
sensor.y = player.y + player.h / 2;
sensor.w = player.w;
sensor.h = 2;
sensor.mass = 0.01;
sensor.removeColliders();
sensor.visible = false;
let sensorJoint = new GlueJoint(player, sensor);
sensorJoint.visible = false;

// --- BACKGROUND  ---
bgLayers = [
  { img: bgFarImg, speed: 0.2 },
  { img: bgMidImg, speed: 0.4 },
  { img: bgForeImg, speed: 0.6 },
];
```

```
  world.autoStep = false;
}

function draw() {
  background(69, 61, 79);

  // --- DEBUG MENU TOGGLE LOGIC ---
  if (kb.presses('`')) {
    debugMenuOpen = !debugMenuOpen;
  }

  // Handle Input for Debug Menu
  if (debugMenuOpen) {
    if (kb.presses('1')) {
      dbgShowStats = !dbgShowStats;
    }
    if (kb.presses('2')) {
      dbgShowCollisions = !dbgShowCollisions;
      allSprites.debug = dbgShowCollisions;
    }
    if (kb.presses('3')) {
      dbgInvincible = !dbgInvincible;
    }
  }

  if (gameState === "PLAYING") {
    world.step();

    // --- TIMER LOGIC ---
    if (!timerStarted) {
      if (kb.pressing("left") || kb.pressing("right") || kb.presses("up")) {
        timerStarted = true;
        lastTick = millis();
      }
    } else {
      if (millis() - lastTick >= 1000) {
        if (!dbgInvincible && timeLeft > 0) {
          timeLeft--;
        }
        lastTick = millis();
        let timeDisplay = dbgInvincible ? "∞" : timeLeft + "s";
        document.getElementById("time-span").innerText = timeDisplay;
      }
    }

    // --- CAMERA ---
    camera.width = VIEWW;
    camera.height = VIEWH;

    let targetX = constrain(
      player.x,
      VIEWW / 2,
```

```
  LEVELW - VIEWW / 2 - TILE_W / 2,
);
let targetY = constrain(
  player.y,
  VIEWH / 2 - TILE_H * 2,
  LEVELH - VIEWH / 2 - TILE_H,
);

camera.x = Math.round(lerp(camera.x || targetX, targetX, 0.1));
camera.y = Math.round(lerp(camera.y || targetY, targetY, 0.1));

// --- PLAYER CONTROLS ---
let grounded =
  sensor.overlapping(ground) ||
  sensor.overlapping(platformsL) ||
  sensor.overlapping(platformsR);

// -- ATTACK INPUT --
if (grounded && !attacking && kb.presses("space")) {
  attacking = true;
  attackFrameCounter = 0;
  player.vel.x = 0;
  player.ani.frame = 0;
  player.ani = "attack";
  player.ani.play();
}

// -- JUMP --
if (grounded && kb.presses("up")) {
  player.vel.y = -4.5;
  jumpSound.play();
}

// --- STATE MACHINE ---
if (attacking) {
  attackFrameCounter++;
  if (attackFrameCounter > 12) {
    attacking = false;
    attackFrameCounter = 0;
  }
} else if (!grounded) {
  player.ani = "jump";
  player.ani.frame = player.vel.y < 0 ? 0 : 1;
} else {
  player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
}

// --- MOVEMENT ---
let isMoving = false;
if (!attacking) {
  player.vel.x = 0;
  if (kb.pressing("left")) {
    player.vel.x = -1.5;
```

```
    player.mirror.x = true;
    isMoving = true;
  } else if (kb.pressing("right")) {
    player.vel.x = 1.5;
    player.mirror.x = false;
    isMoving = true;
  }
}

// --- RUNNING AUDIO LOGIC ---
if (grounded && isMoving && !attacking) {
  if (!runSound.isPlaying()) {
    runSound.loop();
  }
} else {
  if (runSound.isPlaying()) {
    runSound.stop();
  }
}

// --- PLAYER BOUNDS ---
player.x = constrain(player.x, FRAME_W / 2, LEVELW - FRAME_W / 2);

// --- GAME END/WIN STATES ---
if (player.overlaps(treasure)) {
  if (runSound.isPlaying()) runSound.stop();

  uiDiv.hide();
  endDiv.style("display", "flex");

  if (currentLevel === 1) {
    gameState = "WIN_LEVEL";
    document.getElementById("end-title").innerText = "LEVEL 1 CLEAR";
    document.getElementById("end-title").style.color = "#00ff00";
    document.getElementById("end-subtitle").innerText = "Press C for Level 2";
  } else {
    gameState = "WIN_GAME";
    document.getElementById("end-title").innerText = "YOU WIN!";
    document.getElementById("end-title").style.color = "#FFD700"; // Gold
    document.getElementById("end-subtitle").innerText = "Press R to play again";
  }

} else if (timeLeft <= 0 || player.y > LEVELH + TILE_H * 3) {

  if (dbgInvincible && player.y > LEVELH + TILE_H * 3) {
    player.x = FRAME_W;
    player.y = PLAYER_START_Y;
    player.vel.x = 0;
    player.vel.y = 0;
    fallSound.play();
  } else if (!dbgInvincible) {
    if (player.y > LEVELH + TILE_H * 3) {
      fallSound.play();
```

```
      }
      gameState = "LOSE";
      if (runSound.isPlaying()) runSound.stop();

      uiDiv.hide();
      endDiv.style("display", "flex");
      document.getElementById("end-title").innerText = "YOU LOSE";
      document.getElementById("end-title").style.color = "#ff0000";
      document.getElementById("end-subtitle").innerText = "Press R to restart";
    }
  }
} else if (gameState === "WIN_LEVEL") {
  // Wait for player to press C to continue to Level 2
  if (kb.presses("c")) {
    resetLevel(false, 2);
  }
} else {
  // Listen for restart when the game is completely over (Lose or Win Game)
  if (kb.presses("r")) {
    resetLevel(false, 1);
  }
}

// --- BACKGROUNDS ---
camera.off();
imageMode(CORNER);
drawingContext.imageSmoothingEnabled = false;

for (const layer of bgLayers) {
  const img = layer.img;
  const w = img.width;
  let x = Math.round((-camera.x * layer.speed) % w);

  if (x > 0) x -= w;
  for (let tx = x; tx < VIEWW + w; tx += w) {
    image(img, tx, 0);
  }
}
camera.on();

// --- PIXEL SNAP ---
const px = player.x,
  py = player.y;
const sx = sensor.x,
  sy = sensor.y;

player.x = Math.round(player.x);
player.y = Math.round(player.y);
sensor.x = Math.round(sensor.x);
sensor.y = Math.round(sensor.y);

allSprites.draw();
```

```
player.x = px;
player.y = py;
sensor.x = sx;
sensor.y = sy;

// --- HUD STATS OVERLAY ---
if (dbgShowStats) {
  camera.off();
  push();
  fill(0, 150);
  noStroke();
  rect(0, 0, 140, 105);

  fill(255);
  textFont('monospace');
  textSize(10);
  textAlign(LEFT, TOP);

  let grounded = sensor.overlapping(ground) ||
                 sensor.overlapping(platformsL) ||
                 sensor.overlapping(platformsR);

  let debugText =
    `FPS: ${Math.round(frameRate())}\n` +
    `State: ${gameState}\n` +
    `Level: ${currentLevel}\n` +
    `Time: ${dbgInvincible ? '∞' : timeLeft + 's'}\n` +
    `Pos: x:${Math.round(player.x)} y:${Math.round(player.y)}\n` +
    `Vel: x:${player.vel.x.toFixed(1)} y:${player.vel.y.toFixed(1)}\n` +
    `Grounded: ${grounded}`;

  text(debugText, 5, 5);
  pop();
  camera.on();
}

// --- FULL DEBUG MENU OVERLAY ---
if (debugMenuOpen) {
  camera.off();

  push();
  rectMode(CENTER);
  fill(0, 0, 0, 220);
  stroke(100);
  strokeWeight(2);
  // Adjusted height to 160 and shifted down to fit the text better
  rect(VIEWW / 2, VIEWH / 2 + 5, 230, 160, 8);

  // Menu Title
  textAlign(CENTER, CENTER);
  fill(255, 255, 0);
  noStroke();
  textFont('"VT323", monospace');
```

```
    textSize(24);
    text("DEBUG MENU", VIEWW / 2, VIEWH / 2 - 45);

    // Menu Options
    textSize(14);
    textAlign(LEFT, CENTER);
    let startX = VIEWW / 2 - 80;
    let startY = VIEWH / 2 - 15;
    let spacing = 20;

    fill(255);
    text(`[1] Show HUD Stats:    ${dbgShowStats ? "ON" : "OFF"}`, startX, startY);
    text(`[2] Show Collisions:   ${dbgShowCollisions ? "ON" : "OFF"}`, startX, startY + spacing);
    text(`[3] Player Invincible: ${dbgInvincible ? "ON" : "OFF"}`, startX, startY + spacing * 2);

    // Footer Instructions
    textAlign(CENTER, CENTER);
    fill(150);
    textSize(12);
    text("Press 1-3 to toggle, ` to close", VIEWW / 2, VIEWH / 2 + 60);

    pop();
    camera.on();
  }
}

// Added an optional levelNum parameter to force a specific level
function resetLevel(playFallSound, levelNum = currentLevel) {
  player.x = FRAME_W;
  player.y = PLAYER_START_Y;
  player.vel.x = 0;
  player.vel.y = 0;

  if (playFallSound) {
    fallSound.play();
  }

  // Set level and determine correct time limit
  currentLevel = levelNum;
  maxTime = (currentLevel === 1) ? 30 : 15;
  timeLeft = maxTime;
  timerStarted = false;
  gameState = "PLAYING";

  // Reset HTML UI
  if (uiDiv) {
    uiDiv.show();
    document.getElementById("level-span").innerText = "Level " + currentLevel;
    document.getElementById("time-span").innerText = dbgInvincible ? "∞" : timeLeft + "s";
  }
  if (endDiv) endDiv.style("display", "none");

  let spawn = random(spawnPoints);
```

```
  treasure.x = spawn.c * TILE_W + TILE_W / 2;
  treasure.y = spawn.r * TILE_H + TILE_H / 2;
}

function applyIntegerScale() {
  const c = document.querySelector("canvas");
  const scale = Math.max(
    1,
    Math.floor(Math.min(window.innerWidth / VIEWW, window.innerHeight / VIEWH)),
  );
  c.style.width = VIEWW * scale + "px";
  c.style.height = VIEWH * scale + "px";
}
```